Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025 - IB

Lab. 12.1. Algoritmos de Ordenación. Parte II.

Noviembre 05, 2025

Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Los tres algoritmos de ordenación vistos previamente tienen una complejidad de $O(n^2)$. Hay muchas variaciones de estos algoritmos con muy poca ganancia en velocidad, ellos también llegan a tener una complejidad de $O(n^2)$ en el peor de los casos y en el intermedio.

Sin embargo, existen algoritmos que son un poco más rápidos cuya complejidad es O(n log n). El secreto de estos mejores algoritmos es la estrategia divide y vencerás. Esto es, cada algoritmo encuentra una manera de partir la lista en sublistas más pequeñas. Estas sublistas luego son ordenadas recursivamente. Idealmente, si el número de estas subdivisiones es log(n) y la cantidad de trabajo necesario para ordenar los datos en cada subdivisión es n, entonces la complejidad total de tal algoritmo de ordenación es O(n log n). En la siguiente tabla se puede ver que la tasa de crecimiento del trabajo de un algoritmo O(n log n) es más pequeña que un algoritmo $O(n^2)$

n	*n* Log *n*	n^2
512	4,608	262,144
1,024	10,240	1,048,576
2,048	22,458	4,194,304
8,192	106,496	67,108,864
16,384	229,376	268,435,456
32,768	491,520	1,073,741,824

En este laboratorio veremos dos algoritmos de ordenación recursivos que rompen la barrera de n^2: `QuickSort` y `MergeSort`.

El Algoritmo de ordenación QuickSort

`Quicksort` está basado en el enfoque divide y vencerás, se elige un elemento como pivote y se divide la lista alrededor del él: el lado izquierdo del pivote contiene todos los elementos que sean menores que el pivote, el lado derecho contiene todos los elementos mayores que el pivote.
Al seleccionar un pivote al azar resulta en una mejora de la complejidad en la mayoría de los casos.

Implementación del algoritmo QuickSort: Versión 1.

```
# This function takes last element as pivot, places
# the pivot element at its correct position in sorted
# array, and places all smaller (smaller than pivot)
# to left of pivot and all greater elements to right
# of pivot
def partition(arr,low,high):
    i = ( low-1 )         # index of smaller element
    pivot = arr[high]     # pivot

    for j in range(low , high):

        # If current element is smaller than or
        # equal to pivot
        if    arr[j] <= pivot:

            # increment index of smaller element
            i = i+1
            arr[i],arr[j] = arr[j],arr[i]

    arr[i+1],arr[high] = arr[high],arr[i+1]
    return ( i+1 )

# Function to do Quick sort
def quickSort(arr,low,high):
    if low < high:

        # pi is partitioning index, arr[p] is now
        # at right place
        pi = partition(arr,low,high)

        # Separately sort elements before
        # partition and after partition
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)
```

Implementación del algoritmo QuickSort: Versión 2.

```
def quickSort(data_list):
   quickSortHlp(data_list,0,len(data_list)-1)

def quickSortHlp(data_list,first,last):
   if first < last:

       splitpoint = partition(data_list,first,last)

       quickSortHlp(data_list,first,splitpoint-1)
       quickSortHlp(data_list,splitpoint+1,last)


def partition(data_list,first,last):
   pivotvalue = data_list[first]

   leftmark = first+1
   rightmark = last

   done = False
   while not done:
```

```
        while leftmark <= rightmark and data_list[leftmark] <= pivotvalue:
            leftmark = leftmark + 1

        while data_list[rightmark] >= pivotvalue and rightmark >= leftmark:
            rightmark = rightmark -1

        if rightmark < leftmark:
            done = True
        else:
            temp = data_list[leftmark]
            data_list[leftmark] = data_list[rightmark]
            data_list[rightmark] = temp

    temp = data_list[first]
    data_list[first] = data_list[rightmark]
    data_list[rightmark] = temp


    return rightmark
```

1. ¿Cuál es la diferencia entre las dos versiones del algoritmo mostrados? Fíjese que la primera versión usa solamente un bucle en la función `partition` mientras que la segunda usa tres bucles. ¿Se puede decir que la primera versión es mejor?

2. La primera versión escoge de pivote al último elemento, mientras que la segunda escoge al primer elemento. Implemente una nueva versión que escoja de pivote al elemento que se encuentra en la mitad.

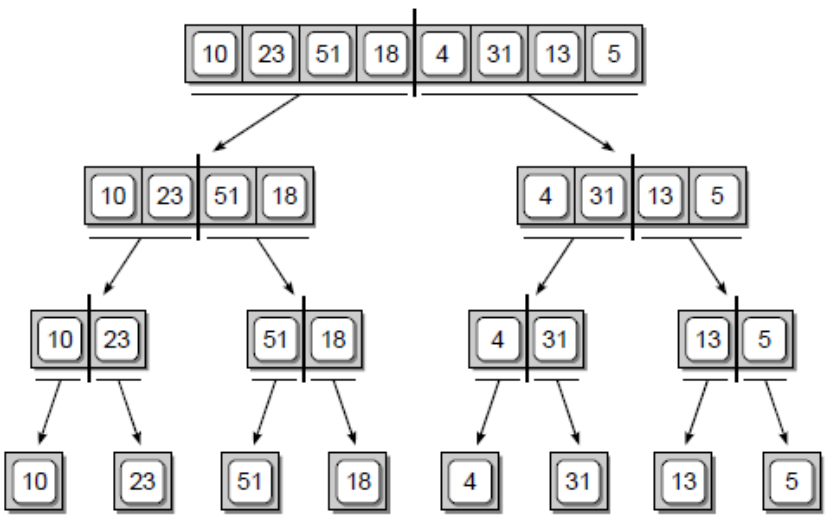
3. ¿Cuál es la complejidad del algoritmo QuickSort en el mejor caso, peor caso y caso medio?

4. ¿Por qué QuickSort no es O(n log n) en todos los casos?. Describa la situación de peor caso para QuickSort y proporcione una lista de 10 enteros, 1-10, que produzca este comportamiento.

Algoritmo de Ordenación MergeSort

El algoritmo llamado **MergeSort** usa también la estrategia divide y vencerás recursivamente a fin de romper la barrera de $O(n^2)$

La lista de valores es dividida recursivamente en sublistas cada vez más pequeñas hasta que cada valor esté contenido en su propia sublista. Las sublistas luego se juntan (merge) para crear una lista ordenada.



Implementación del Algoritmo MergeSort

```
def mergeSort(lyst):
    # lyst list being sorted
    # copyBuffer temporary space needed during merge
    copyBuffer = list(lyst)
    mergeSortHelper(lyst, copyBuffer, 0, len(lyst) - 1)

def mergeSortHelper(lyst, copyBuffer, low, high):
    # lyst list being sorted
    # copyBuffer temporary space needed during merge
    # low, high bounds of sublist
    # middle midpoint of sublist
    if low < high:
        middle = (low + high) // 2
        mergeSortHelper(lyst, copyBuffer, low, middle)
        mergeSortHelper(lyst, copyBuffer, middle + 1, high)
        merge(lyst, copyBuffer, low, middle, high)

def merge(lyst, copyBuffer, low, middle, high):
    # lyst list that is being sorted
    # copyBuffer temp space needed during the merge process
    # low beginning of first sorted sublist
    # middle end of first sorted sublist
    # middle + 1 beginning of second sorted sublist
    # high end of second sorted sublist

    # Initialize i1 and i2 to the first items in each sublist
    i1 = low
    i2 = middle + 1
    # Interleave items from the sublists into the
    # copyBuffer in such a way that order is maintained.
    for i in range(low, high + 1):
        if i1 > middle:
            copyBuffer[i] = lyst[i2] # First sublist exhausted
            i2 += 1

        elif i2 > high:
            copyBuffer[i] = lyst[i1] # Second sublist exhausted
            i1 += 1
        elif lyst[i1] < lyst[i2]:
            copyBuffer[i] = lyst[i1] # Item in first sublist <
            i1 += 1
        else:
            copyBuffer[i] = lyst[i2] # Item in second sublist <
            i2 += 1
    for i in range(low, high + 1): # Copy sorted items back to
        lyst[i] = copyBuffer[i] # proper positions in lyst
```

5. ¿Cuál es la complejidad del algoritmo MergeSort en el caso peor, caso mejor e intermedio?.

6. ¿Por qué MergeSort es un algoritmo O(n log n) en el peor caso?

7. Dada la siguiente lista de números: [21, 1, 26, 45, 29, 28, 2, 9, 16, 49, 39, 27, 43, 34, 46, 40] ¿cuál de las respuestas ilustra la lista a ser ordenada después de 3 llamadas recursivas a MergeSort?

   a. [16, 49, 39, 27, 43, 34, 46, 40]
   b. [21, 1]
   c. [21, 1, 26, 45]
   d. [21]

8. Dada la siguiente lista de números: [21, 1, 26, 45, 29, 28, 2, 9, 16, 49, 39, 27, 43, 34, 46, 40] ¿cuál de las respuestas ilustra las dos primeras listas a ser concatenadas?.

   a. [21, 1] y [26, 45]
   b. [[1, 2, 9, 21, 26, 28, 29, 45] y [16, 27, 34, 39, 40, 43, 46, 49]
   c. [21] y [1]
   d. [9] y [16]

OPCIONAL

9. Implementar un programa que ordene una lista enlazada usando MergeSort.

Guarde todos vuestros programas en una carpeta con el nombre su **Apellido** paterno seguido de vuestro **DNI**, luego comprima esta carpeta. Envíe este archivo a Katherine Navarro katherine.navarro@upch.pe especificando como asunto **Lab12.1**.